



Lesson 1: Introduction and the ML Workflow

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi
dell'Aquila

Welcome

- 10 lessons, 3 hours each, every Friday until 4 December
- A first course in **machine learning**, for people who can already program
- Foundations, not products
- Course team on Microsoft Teams: code **v0t2gvu**

What we will cover

1-2	Workflow, data preparation
3-4	Regression, classification
5	Experimental methodology
6-7	k-NN, support vector machines, trees, ensembles
8	Unsupervised learning
9-10	Neural networks, CNNs

What this course is not

- Not a tour of current AI products
- Not a library tutorial
- Not a course where you call `.fit()` and report the number

Your advantage, and your gap

Advantage: you can program. You will implement methods, not only call them.

Gap: you can produce results faster than you can judge them.

How you will be assessed

- **Weekly exercises:** set each Friday, discussed the next. Never collected
- **Exam: one of the ten is drawn**, and you talk through your own notebook
- No written paper, no project. Keep all ten

The three things you get each lesson

- **Handout:** the reference text, with the mathematics
- **Slides:** what we do in the room
- **Notebooks:** runnable implementations

Plus a quiz, and the exercise.

Getting the material

```
U=https://github.com/fabioantonini  
R=technologies-for-artificial-intelligence  
git clone $U/$R.git tai  
cd tai  
docker compose pull  
docker compose up
```

Then open **127.0.0.1:8888**, token aicourse

Today

- What learning from data means
- A short history, and what it teaches
- The three kinds of learning
- The end-to-end workflow
- Four ways a model misleads you

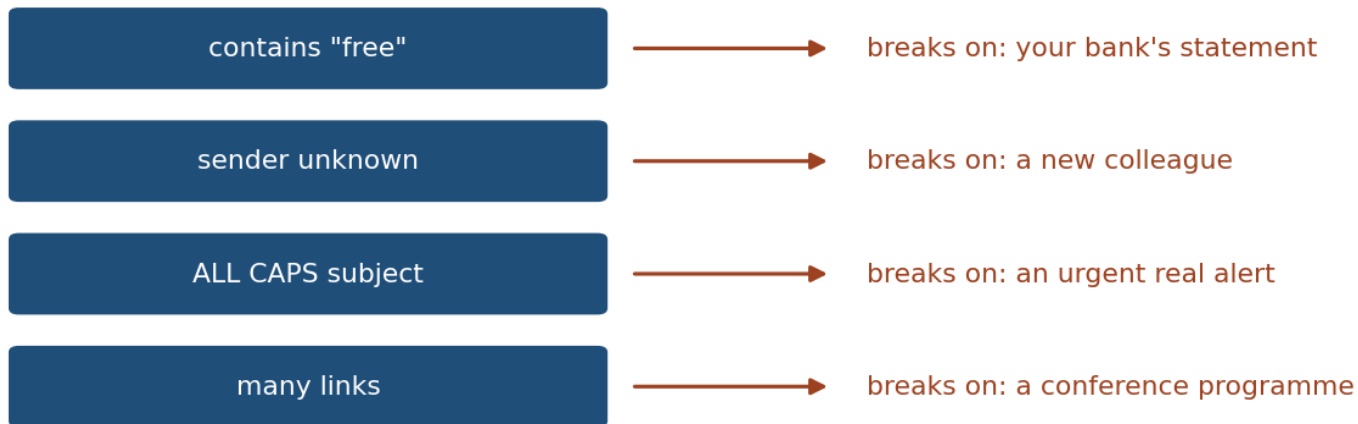
A problem you cannot specify

Write a program that decides whether an email is spam.

- Filter certain words? Spam adapts.
- Filter certain senders? They change.
- The exceptions grow until nobody can maintain it

Every rule buys an exception

Every rule buys an exception

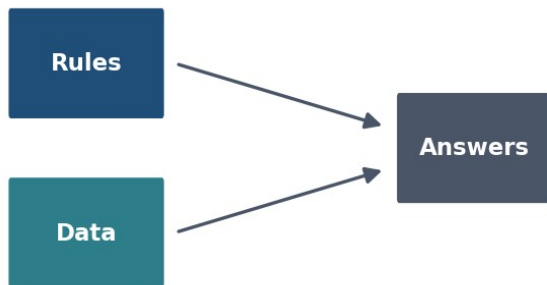


you cannot state the rule, but you recognise the answer

The inversion

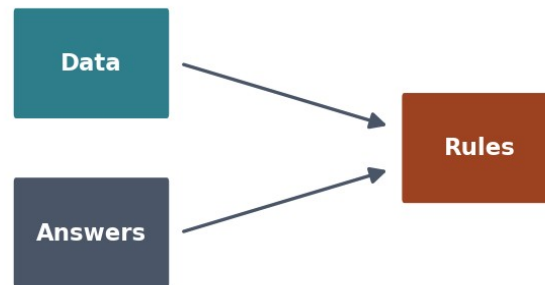
You cannot state the rule. But you can **recognise** the answer.

Traditional programming



you write the rules

Machine learning



the rules are learned

What the data actually is

- One tumour = 30 numbers: radius 17.99, texture 10.38, perimeter 122.80, ...
- That row is one input x ; its label y is `malignant`, coded 0
- All 569 rows stacked is the design matrix X , 569 x 30
- **$m = 569$ examples** (rows), **$n = 30$ features** (columns), all course long

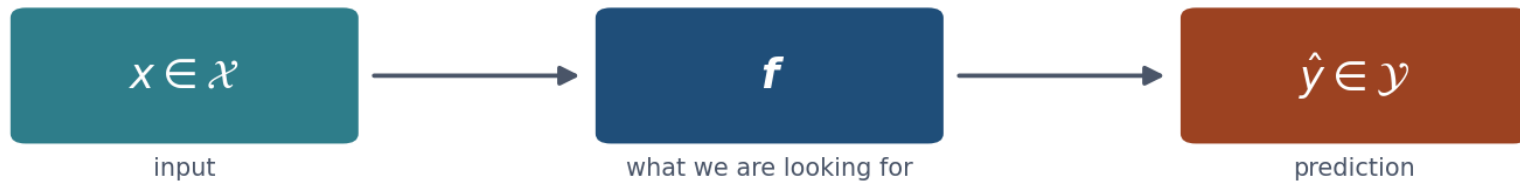
Making it precise

- Inputs from a space X , outputs from a space Y
- Pairs (x, y) drawn from an unknown distribution D
- A loss $L(\hat{y}, y)$: the cost of answering $\hat{y}$ when the truth is y
- Goal: find $f: X \rightarrow Y$ that predicts well

The setup, in one picture

Four objects, one of them unknowable

$$(x, y) \sim \mathcal{D} \quad (\text{unknown})$$



loss $L(\hat{y}, y)$ scores the answer

What we actually want

The average loss over all data the world might produce, including data that does not exist yet:

$$R(f) = \mathbb{E}_{(x, y) \sim \mathcal{D}}[L(f(x), y)]$$

What we can actually compute

Average loss over the finite sample we happen to hold:

$$\hat{R}_S(f) = \frac{1}{m} \sum_{i=1}^m L(f(x_i), y_i)$$

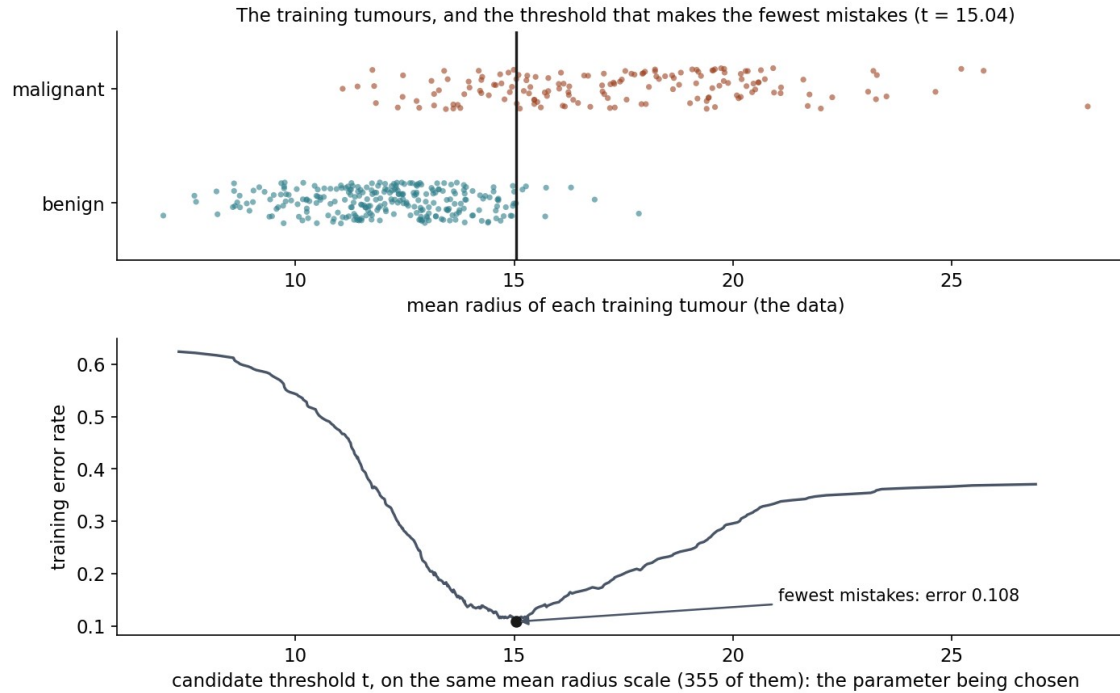
Accuracy: the share of right answers

- With the zero-one loss - 1 for a wrong answer, 0 for a right one - the empirical risk is the **error rate**
- **Accuracy** = right answers $\div$ all answers = 1 – error rate
- 141 right out of 143: accuracy **0.986**, error rate **0.014**
- Every mistake counts the same, which is what we will see it hide

Learning, in one line

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_S(f)$$

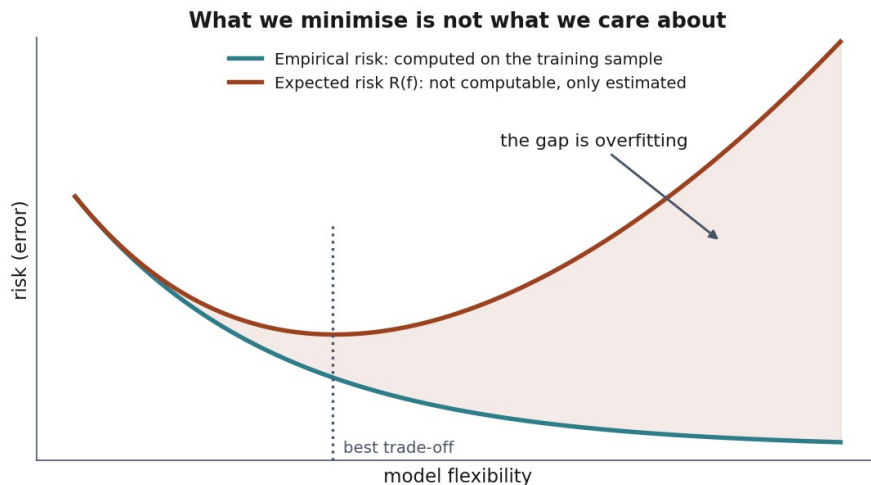
Learning is a search



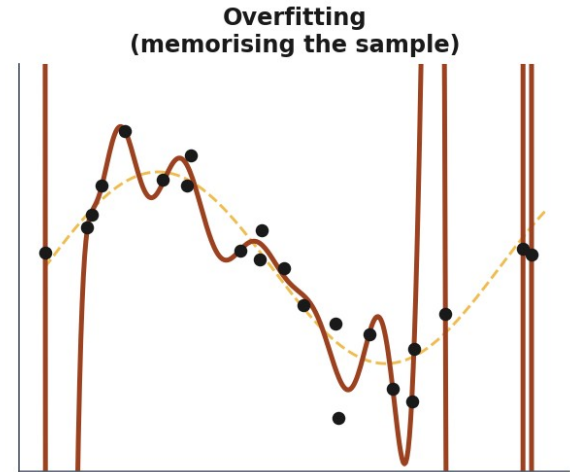
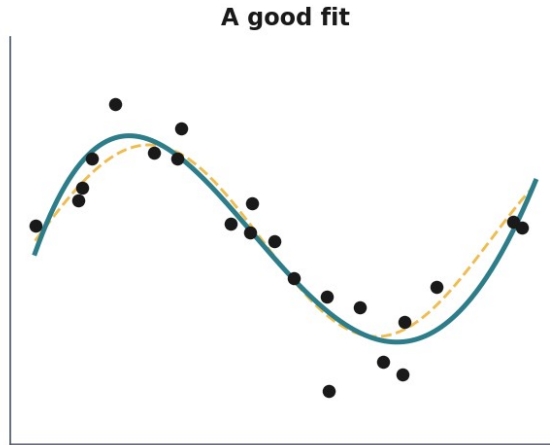
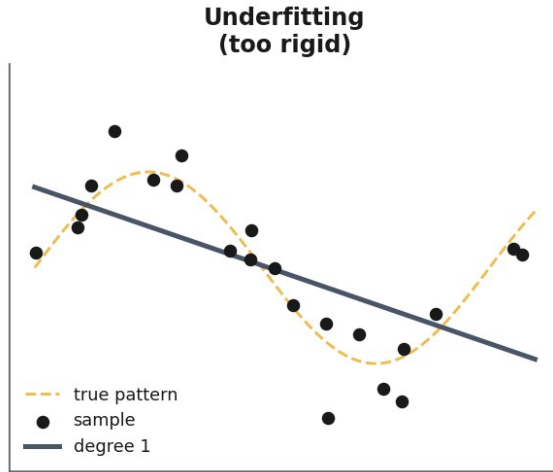
The gap that explains everything

We **minimise** the empirical risk

We **care about** the expected risk



The lowest training error is the worst model



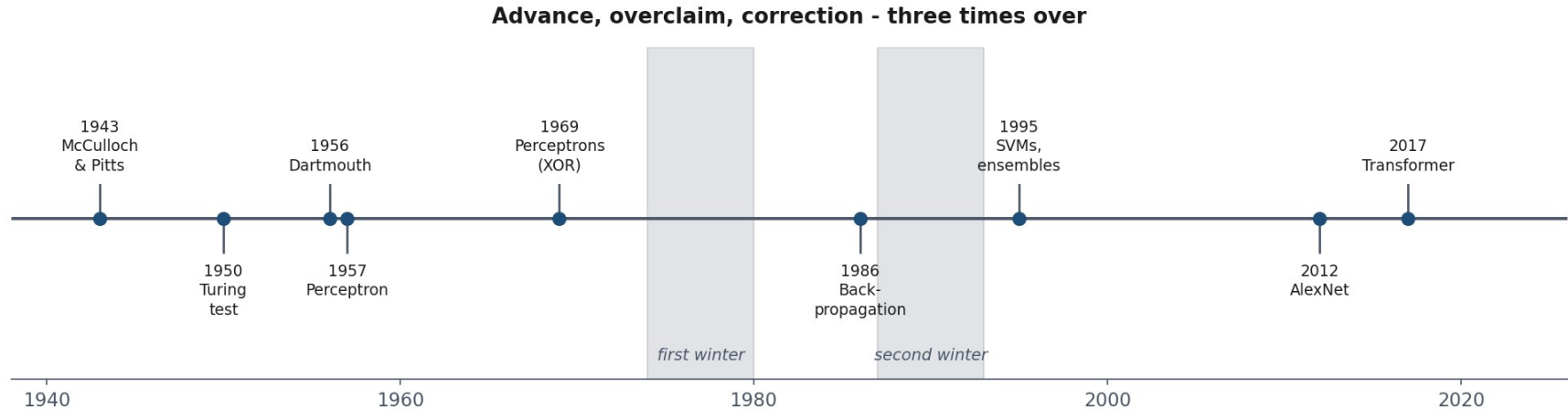
Which gives us the one rule for today

Hold out part of the data. Never let the fitting procedure touch it. Measure there.

When *not* to use machine learning

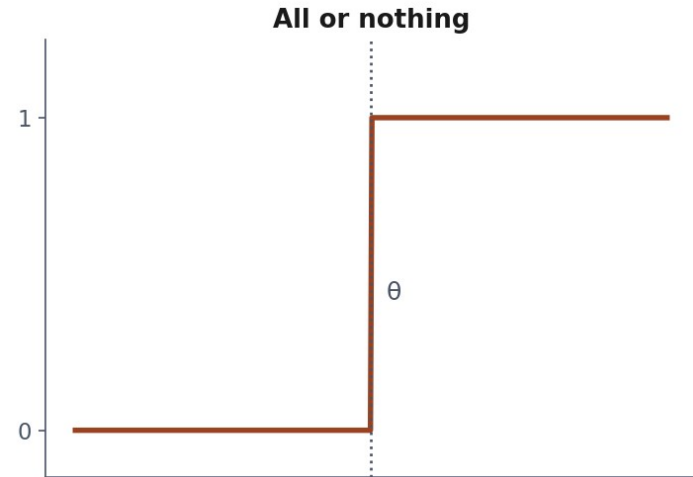
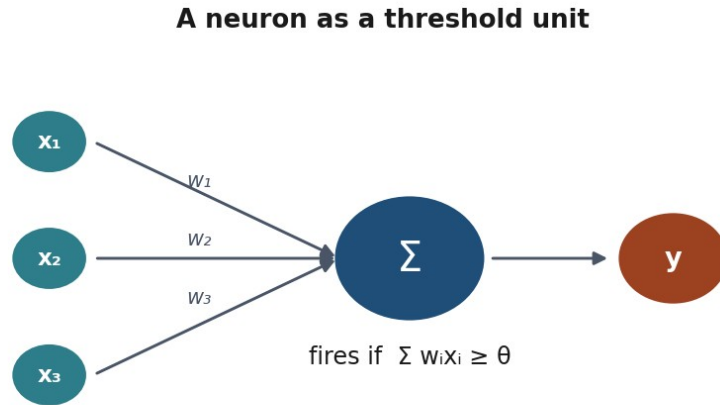
- The rule is **known**: write it
- The data is **not representative** of deployment
- Errors are **certain, unrecoverable and unreviewed**
- The data encodes an **injustice** you would automate
- A **simple baseline** already suffices: always give the most common answer

Seventy years in one picture



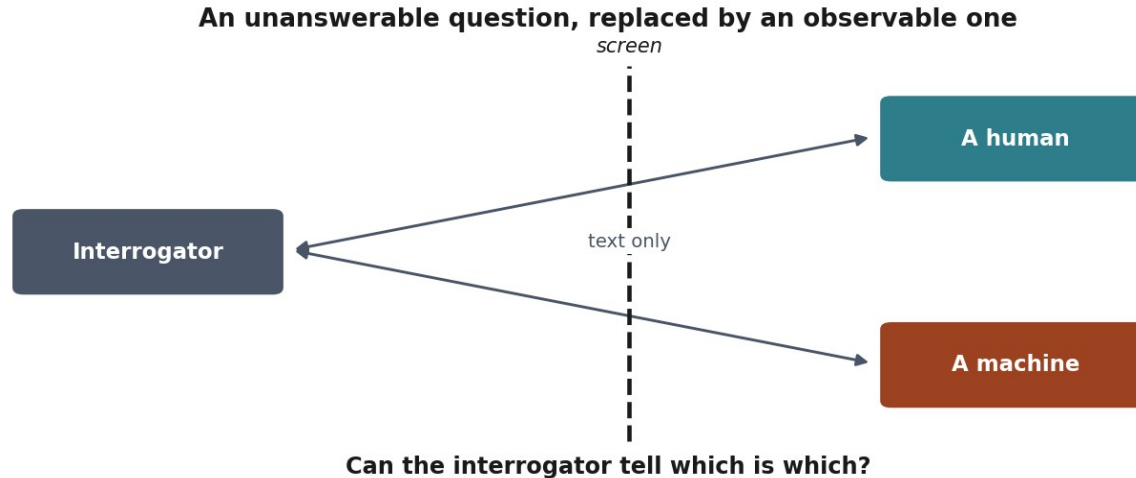
1943: the first artificial neuron

A neuron as a threshold unit: it fires when the weighted sum passes θ .



1950: Turing's imitation game

“Can machines think?” becomes a question you can settle by observation.



1956: Dartmouth

The term *artificial intelligence* is coined.

The seven topics of the Dartmouth proposal

Automatic computers

Language use

Neuron nets

Theory of computation

Self-improvement

Abstractions

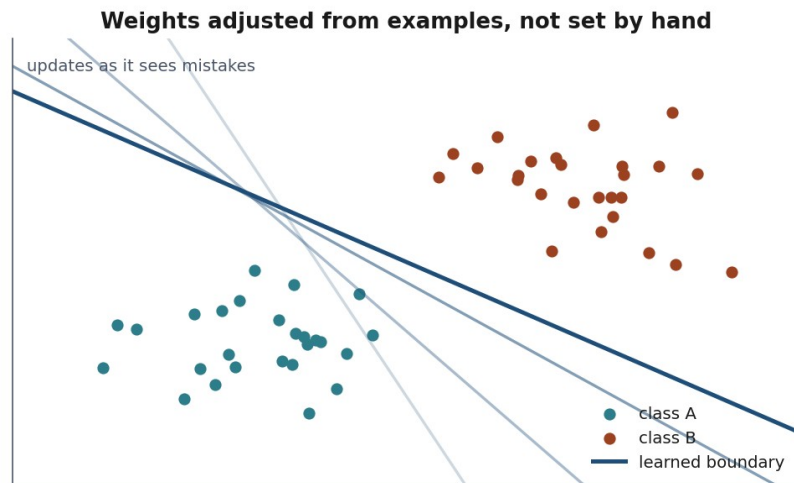
Randomness and creativity

“a 2 month, 10 man study”

Seventy years later,
most of this list
is still open.

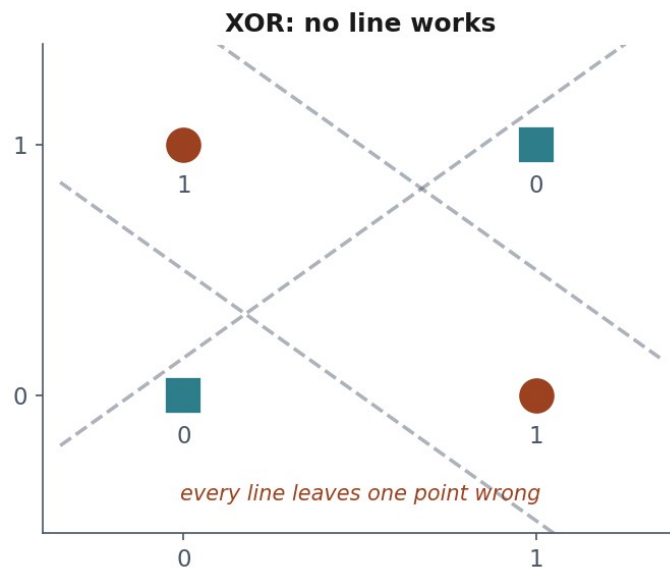
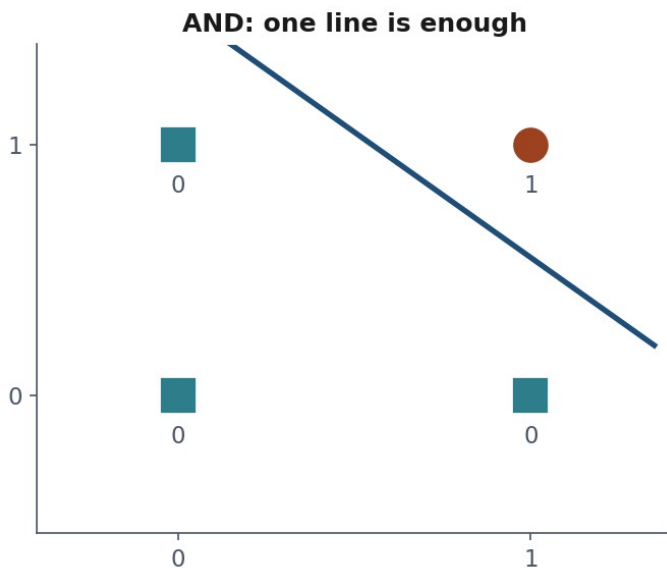
1957: the perceptron learns

Rosenblatt: weights adjusted **from examples**, not set by hand.



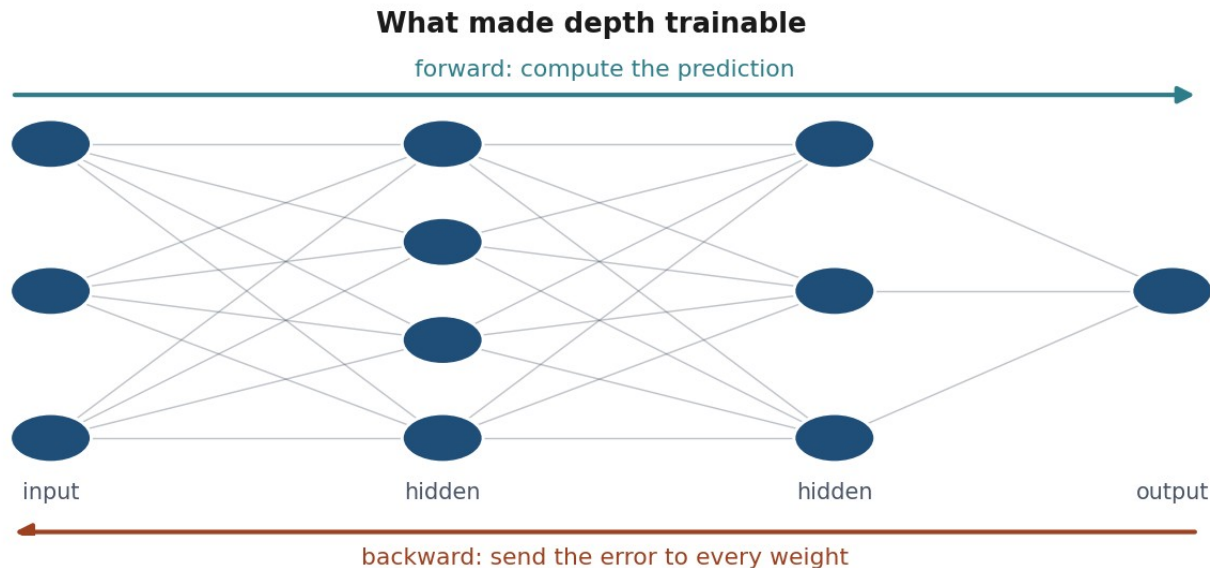
1969: the first winter

A single-layer perceptron cannot represent XOR.



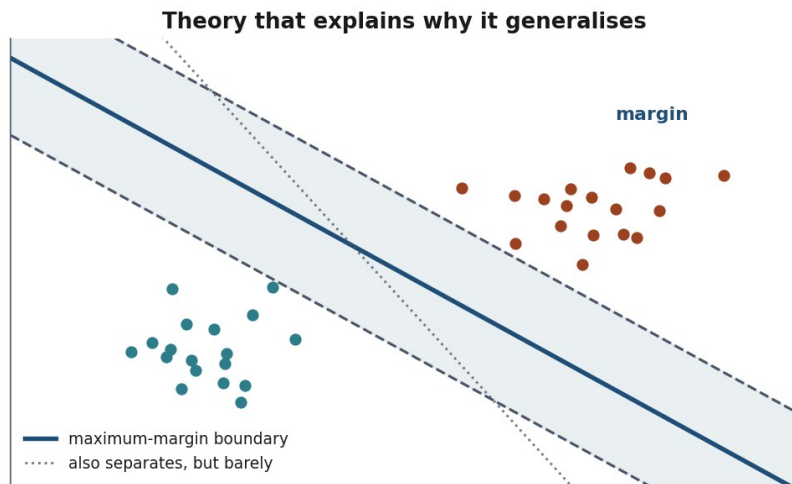
1986: backpropagation

An efficient way to train multi-layer networks.



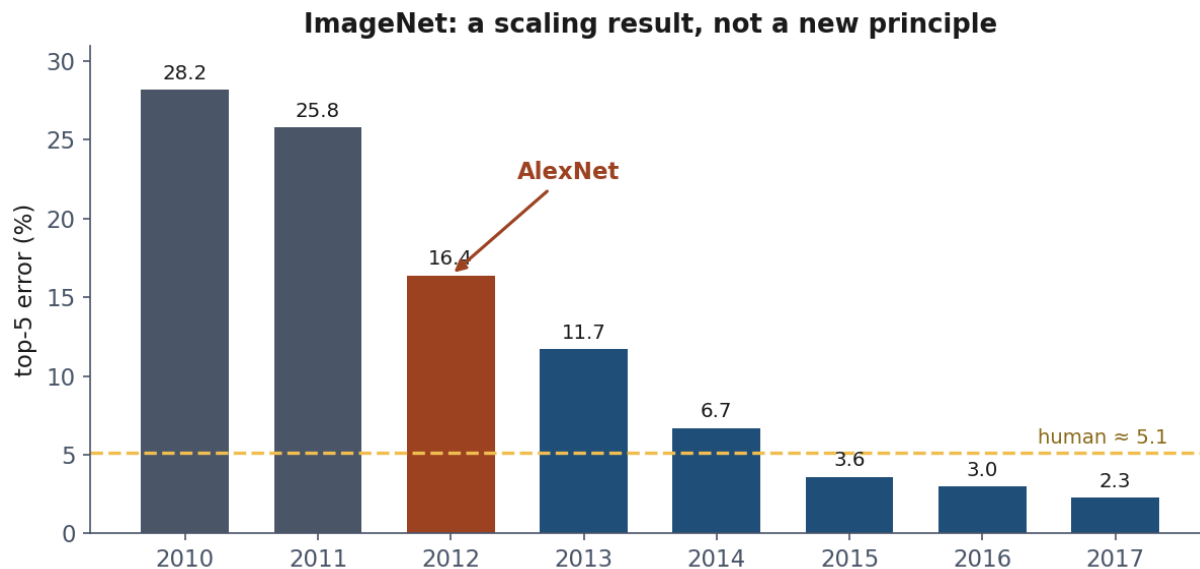
1995-2010: statistics takes over

From “simulating intelligence” to “estimating functions from data”.



2012: AlexNet

The architecture was recognisably LeNet, from 1989.



The pattern

Advance	Overclaim	Correction
Perceptron learns	Machines that walk and talk	XOR, first winter
Backprop trains depth	Networks solve everything	No data, second winter
Scale delivers	?	?

Three kinds of learning

The taxonomy divides by **where the target comes from**, not by algorithm.

Supervised

The answer is recorded by somebody, outside the measurements.

- Category → **classification**
- Continuous → **regression**
- The cultivar is not a measurement: it is an answer written down
- In code: a classifier such as `LogisticRegression`, a regressor such as `LinearRegression`

Unsupervised

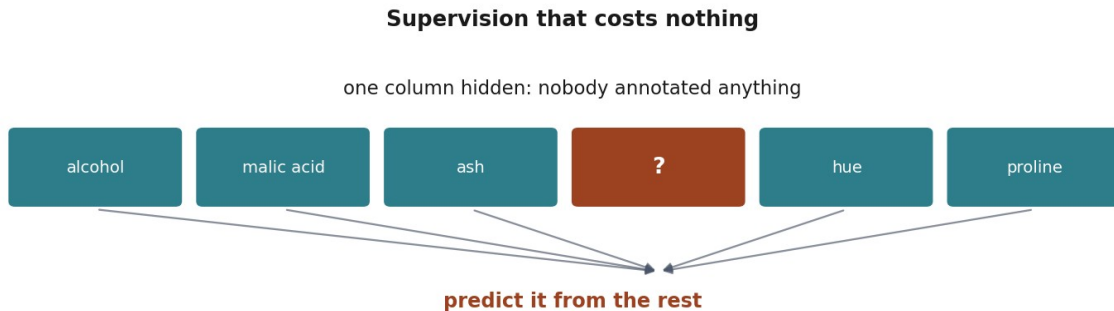
No targets at all. Find structure.

- Groups → clustering
- Fewer dimensions → dimensionality reduction
- Odd points → anomaly detection
- In code: `KMeans` for groups, `PCA` for fewer dimensions

Self-supervised

Hide part of the input. Predict it from the rest.

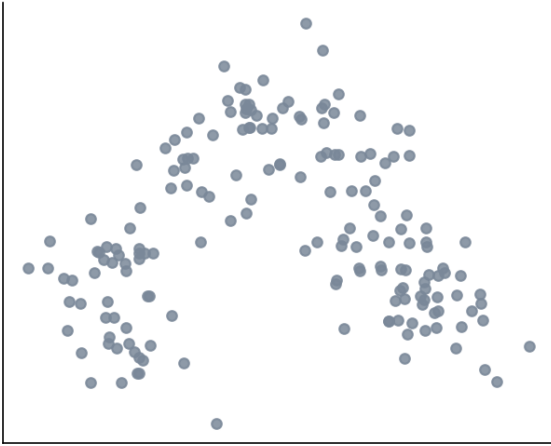
Nobody annotates anything, and the supervision is real.



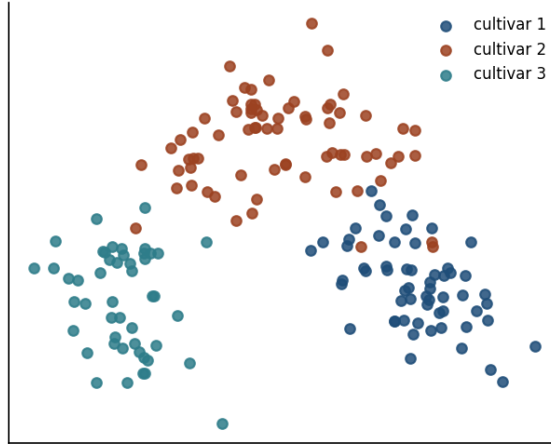
the task is a pretext: what transfers is having learned how the columns relate

One dataset, three questions

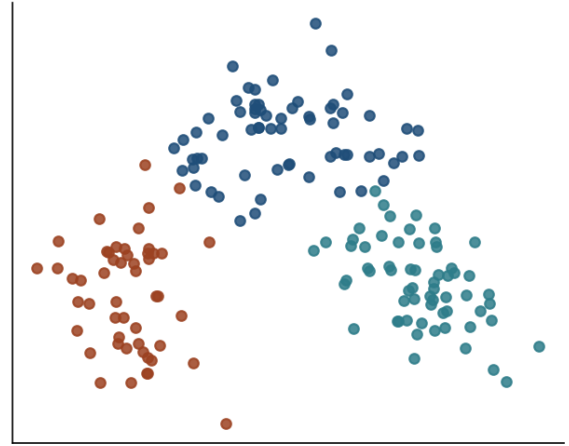
What the algorithm sees
(no labels)



Supervised: labels supplied
by a person



Unsupervised: groups found
by geometry alone



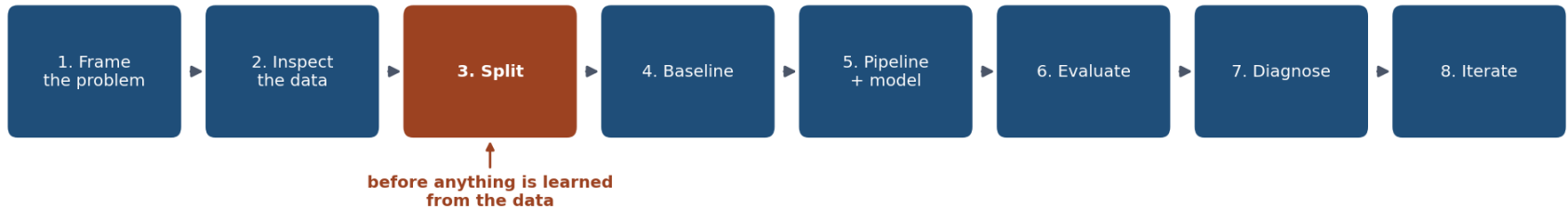
Where the target comes from

	Where the answer comes from	Cost	Measurable?
Supervised	Recorded by somebody	Expensive	Yes
Unsupervised	Nowhere	Free	Not directly
Self-supervised	Part of the input	Free	Only the pretext

Notebook 1, live

question	you call, and what it scores
the data: 178 wines, 13 measurements	<code>load_wine</code>
supervised: predict the cultivar	<code>make_pipeline(StandardScaler(), LogisticRegression())</code> → accuracy_score 0.981
unsupervised: labels thrown away	<code>KMeans(n_clusters=3)</code> , drawn with PCA → adjusted_rand_score 0.897
self-supervised: hide flavanoids, predict it	<code>LinearRegression</code> → r2_score 0.816 (1 perfect, 0 = guessing the mean)
before either model with a target	<code>train_test_split</code>

The workflow



Notebook 2, live

step, and its slide	you call
look, 46	<code>load_breast_cancer</code>
split, 47	<code>train_test_split(X, y, test_size=0.25, stratify=y)</code>
baseline, 48	DummyClassifier, always the majority class
model, 49	<code>make_pipeline(StandardScaler(), LogisticRegression())</code> , then <code>.fit()</code>
result, 51	<code>.score(X_test, y_test)</code>
errors, 52-55	<code>ConfusionMatrixDisplay</code> , <code>classification_report</code> , <code>predict_proba</code>

Step 1: frame it

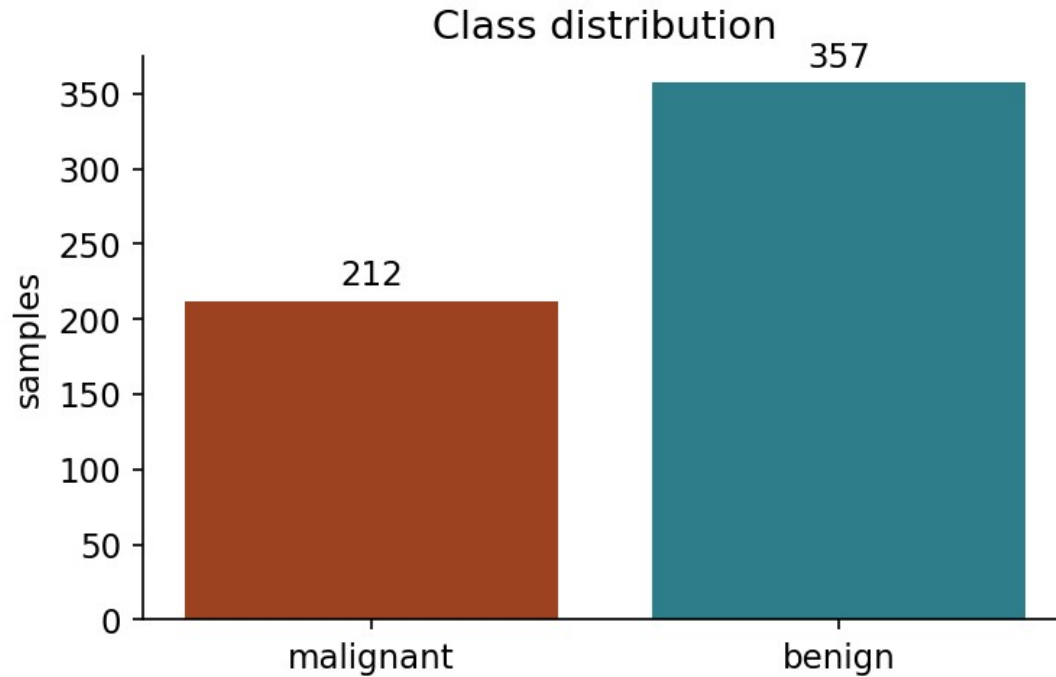
- What is predicted, from what?
- Who would use it, for what decision?
- **Which error is worse?**

Which error is worse?

Accuracy weights these equally. Nobody else does.

	predicted benign	predicted malignant
benign	correct no action needed	false alarm anxiety, a follow-up test
malignant	missed case a patient sent home who should not be	correct treated in time

Step 2: look first



Step 3: split, before anything else

Before scaling. Before selecting features. Before looking at correlations with the target.

Split first, then never look right again until you are finished



```
In code: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y)
```

Step 4: baseline first

Predict the majority class. Nothing else.

62.9% on the test set

Everything from here is measured against that.

In code:

```
DummyClassifier(strategy="most_frequent")
```


Step 5: pipeline, not two steps

```
model = make_pipeline(  
    StandardScaler(),  
    LogisticRegression(max_iter=5000),  
)  
model.fit(X_train, y_train)
```

What the pipeline prevents

Preprocessing outside the split

the mean was computed over the test rows too



optimistic score, no error, nothing to notice

Preprocessing inside the pipeline

the mean is learned from training rows only



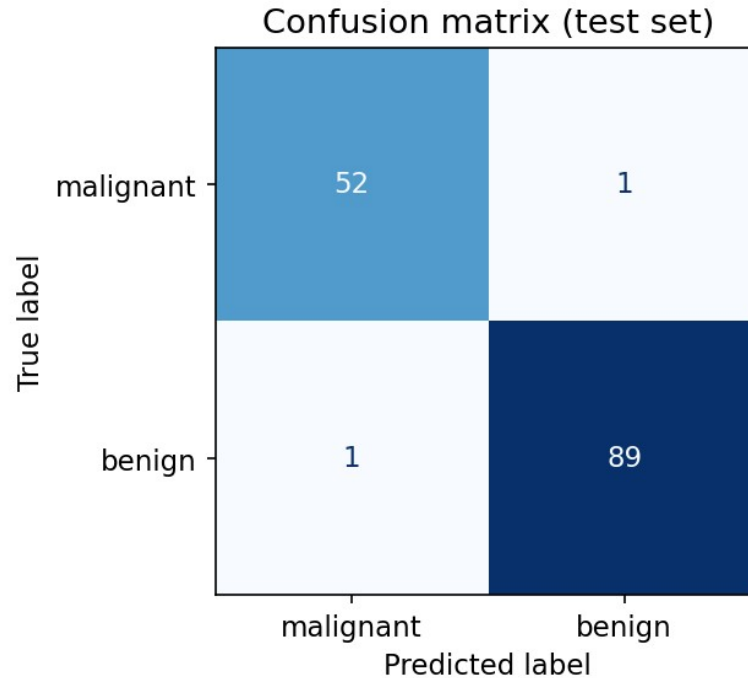
and it is refitted inside every cross-validation fold

Step 6: the result

- Baseline: 0.629
- Model: **0.986**
- In code: `baseline.score(X_test, y_test)`, then
`model.score(X_test, y_test)`

Done?

What accuracy hides



Precision and recall

- **Recall** (malignant): how many malignant tumours we caught
- **Precision** (malignant): how often an alert is correct
- They trade off against each other
- In code: `classification_report(y_test, y_pred)` prints both

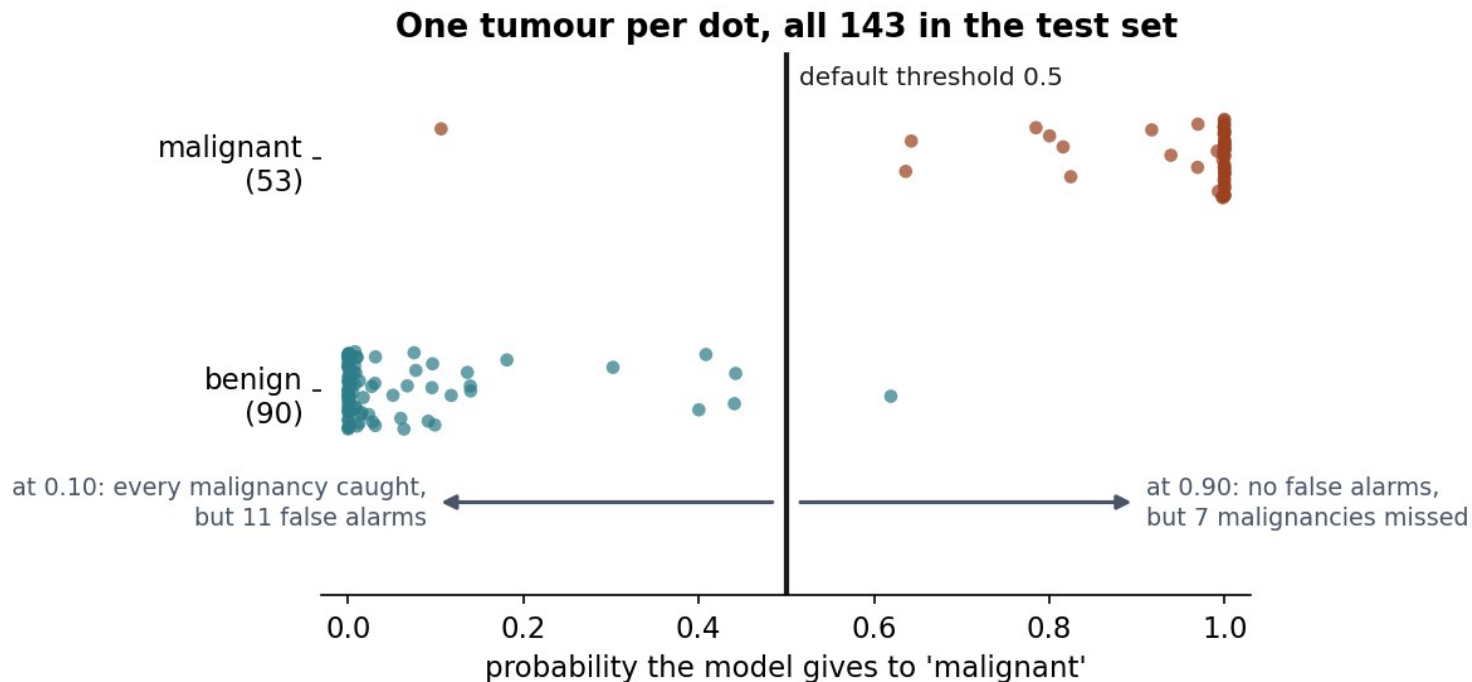
Precision and recall, counted

	what it counts	our model
true positives (TP)	malignant, flagged	52
false negatives (FN)	malignant, missed	1
false positives (FP)	benign, flagged	1
recall = $TP / (TP + FN)$	malignant tumours we caught	$52 / 53 = \mathbf{0.981}$
precision = $TP / (TP + FP)$	alerts that were right	$52 / 53 = \mathbf{0.981}$

Behind every label, a probability

- The model does not output a label: it outputs **the probability** that the tumour is malignant,
`predict_proba()`
- `predict()` turns it into a label by comparing it with **0.5**
- The missed malignancy got **0.11**: called benign
- The false alarm, a benign tumour, got **0.62**: called malignant
- 0.5 is a default. Nobody chose it

The trade-off, drawn



What we did not do

- No tuning
- No comparison of model families
- No check of stability across splits

And: we never looked at the test set to make a decision.

Four ways a model misleads you

- Leakage
- Imbalance
- Shortcut features
- Single-split noise

All produce numbers that pass a casual review.

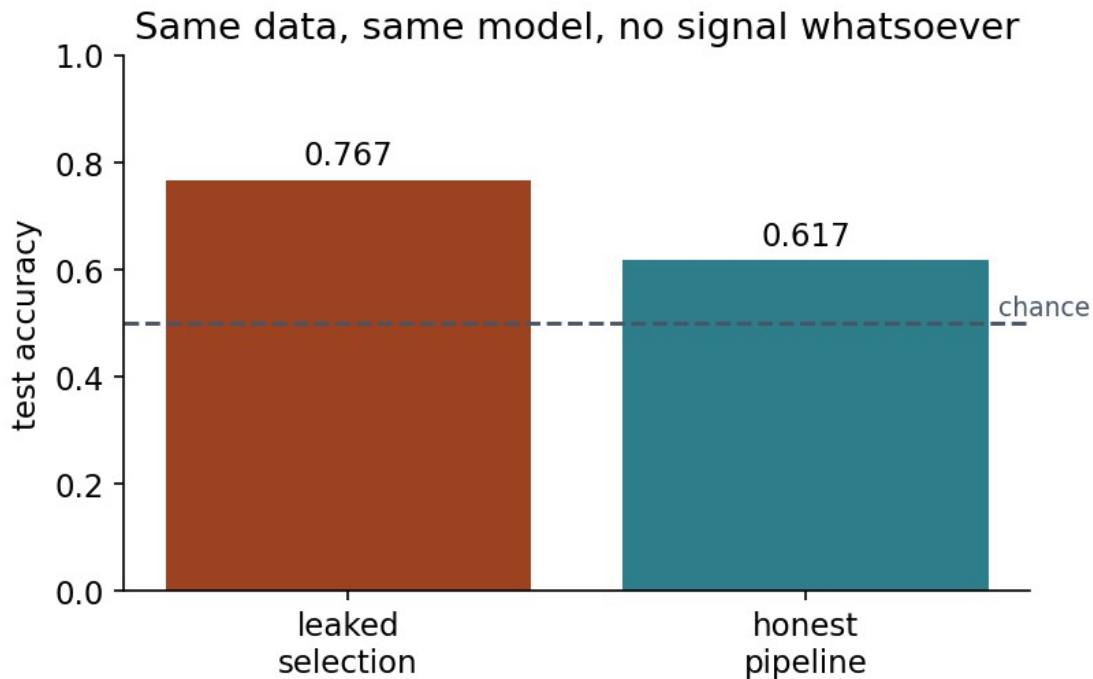
Notebook 3, live

failure, and its slides	you call
the model, every time, 49	<code>make_pipeline(StandardScaler(), LogisticRegression())</code>
leakage, 60-62	<code>SelectKBest(f_classif, k=20)</code> , outside the pipeline and then inside it
imbalance, 63-64	<code>make_classification; accuracy_score</code> against <code>recall_score, confusion_matrix</code>
shortcut, 65-66	<code>load_breast_cancer; SimpleImputer</code> when the column is missing
single-split noise, 67	<code>train_test_split(random_state=seed)</code> against <code>cross_val_score</code>

200 samples. 5000 random features.
Coin-flip labels.

There is nothing to learn.
Anything above 50% is an artefact.

Select features first, split second



The rule

Every step that **learns anything** must be fitted on the training data only.

Scaling. Imputation. Encoding. Selection. Tuning.

In code: `StandardScaler`, `SimpleImputer`,
`SelectKBest...` all inside `make_pipeline`

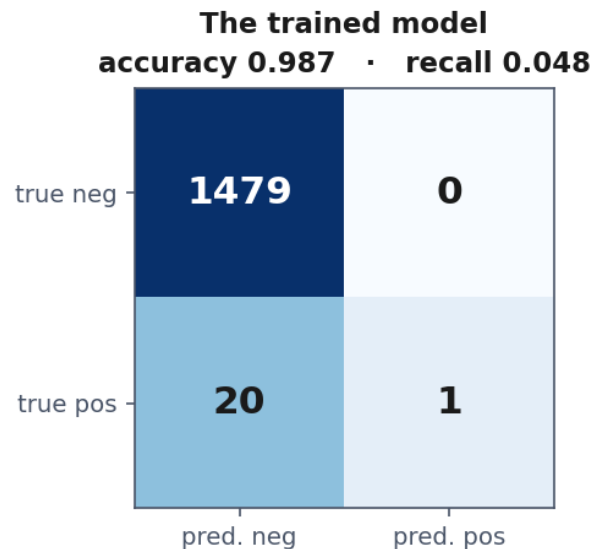
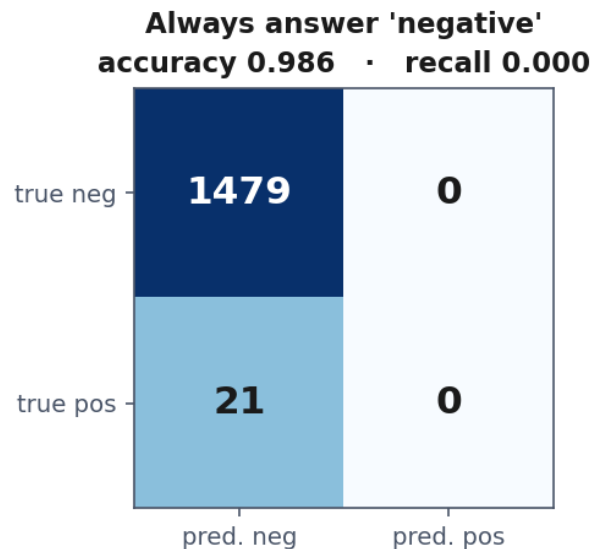
99% accuracy, detecting nothing

1% positives. Answer “negative” every time.

- Accuracy: **0.986**
- Recall on positives: **0.000**
- In code: `recall_score` beside `accuracy_score`, every time

Two models, one accuracy

Nearly identical accuracy. One of them finds nothing.



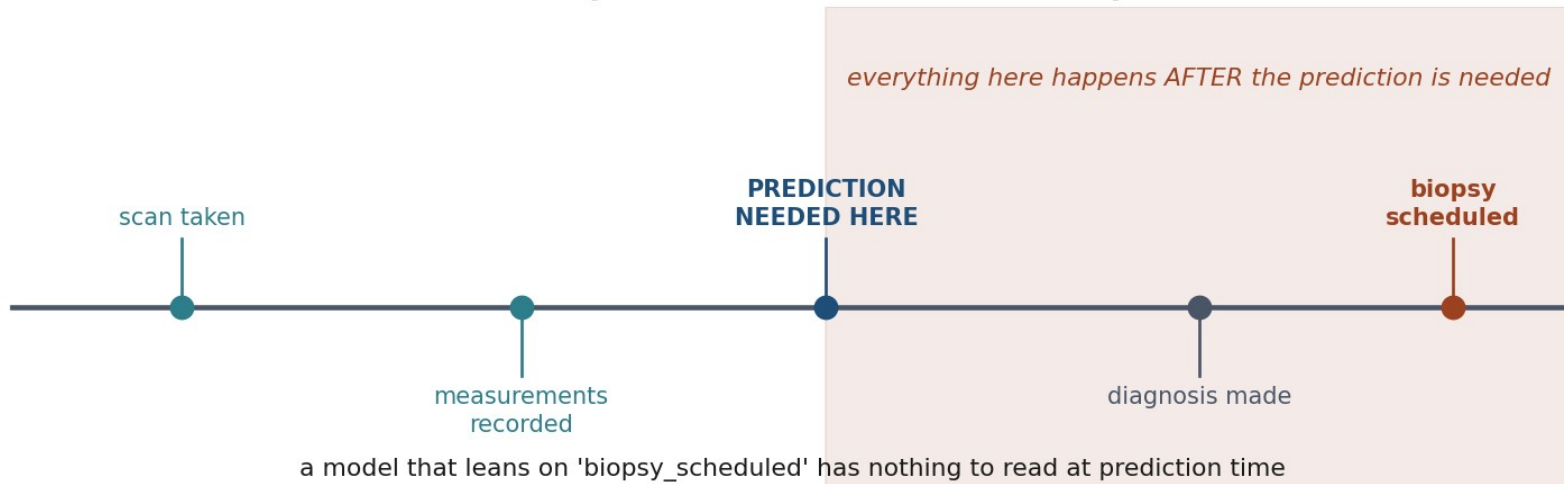
Shortcut features

A column that is a **consequence** of the label, not a predictor of it.

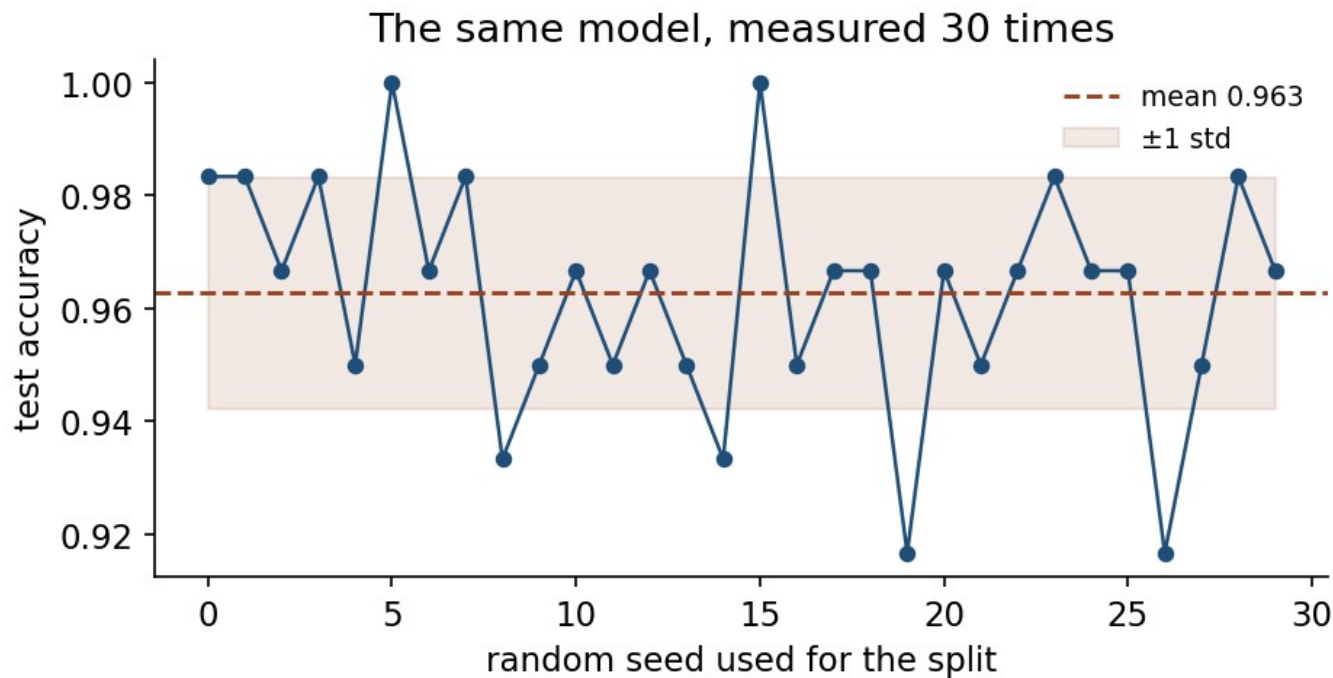
biopsy_scheduled → accuracy rises → model is worthless

Does this value exist yet?

Ask of every feature: does this value exist yet?



One split is one measurement



What the four have in common

None is a coding error.

All four run exactly as written and return a plausible number.

A model is not objective

- It is a compressed summary of its training data, **including its injustices**
- These methods find **correlation**, not cause
- Accuracy is not the only property that matters

Homework: we discuss it on Friday 9 October

Exercises/01_first_workflow.md

Wine quality: run the workflow yourself, and **justify every decision**.

- Accuracy is not what counts
- Methodology is, and saying what your number does not mean

Before next week

- Get the environment running
- Work the three notebooks in order
- Read the handout
- Take the quiz
- **Do the exercise**

Next week: data exploration, preparation, and leakage in earnest.